

# AI Air-Drum Pad: 손 랜드마크 기반 타격 검출을 이용한 비접촉 음악 인터페이스

최종 보고서

저자: FILLME

소속: FILLME

작성일: 2026-05-26

저장소: [git@github.com:GW-arch/deepx.git](https://github.com/GW-arch/deepx.git) 내 `air_drum_pad/`

## 초록

본 보고서는 USB 카메라와 손 랜드마크 추적을 이용해 사용자의 손가락 움직임을 드럼 및 피아노 연주 이벤트로 변환하는 AI Air-Drum Pad 시스템을 설명한다. 시스템은 각 손가락의 손끝 하강 속도와 관절 각속도를 동시에 계산하여 두 조건이 모두 만족될 때만 타격으로 판단한다. 이를 통해 단순한 손 이동이나 팔 흔들림으로 인한 오탐을 줄이고, 실제 악기를 치는 동작에 가까운 입력을 인식한다. 구현된 시스템은 동일한 인식 및 타격 검출 코어 위에서 두 가지 음악 인터페이스를 제공한다. 첫째, 드럼 패드 모드는 화면에 표시된 직사각형 패드 안에서 어떤 손가락이든 내리치면 해당 드럼 소리를 재생한다. 둘째, 피아노 모드는 손과 손가락 ID를 음에 매핑한다. 최종 기본 피아노 배치는 왼손 엄지부터 소지까지 G4, F4, E4, D4, C4, 오른손 엄지부터 소지까지 C5, D5, E5, F5, G5이다. 구현은 CPU MediaPipe, CPU TFLite baseline, CPU+NPU 하이브리드, NPU dual-halves 백엔드를 지원한다. 단위 테스트와 품질 검사 스크립트는 타격 검출, 패드 영역 파싱, 피아노 매핑, ROI 변환, 벤치마크 헬퍼를 검증한다. 프로토타입 측정 결과, NPU dual-halves 모드는 60 FPS 프레임 예산에 근접할 수 있으나, 정확한 palm detection 기반 파이프라인은 CPU palm detection 지연의 영향을 크게 받는다. 본 프로젝트는 엣지 AI 하드웨어에서 동작하는 저비용 비접촉 음악 인터페이스의 가능성을 보이며, 향후 E2E 오디오 지연 측정, 타격 정확도 실험, 사용자 평가가 필요하다.

키워드: 비접촉 음악 인터페이스, 손 추적, 제스처 인식, 에어 드럼, 가상 피아노, 엣지 AI, NPU, 실시간 상호작용

## 1. 서론

전통적인 악기 연주는 물리적 접촉에 기반한다. 드럼 스틱은 드럼 헤드를 치고, 피아노 연주자는 건반을 누른다. 물리적 접촉은 촉각 피드백과 명확한 입력 이벤트를 제공하지만, 별도의 악기 또는 컨트롤러를 요구한다. 반면 카메라 기반 손 추적 기술은 별도 컨트롤러 없이 손 움직임만으로 음악을 연주하는 비접촉 악기 인터페이스를 가능하게 한다. 이러한 시스템은 교육, 접근성, 전시, 임베디드 AI 데모, 저비용 프로토타이핑에 유용하다.

비접촉 음악 인터페이스의 핵심 문제는 “의도적인 타격”을 안정적으로 구분하는 것이다. 단순한 손끝 위치 기반 트리거는 손이 영역을 지나가기만 해도 소리가 날 수 있고, 단순 속도 기반 트리거는 팔 전체가 움직일 때 오탐을 만들 수 있다. AI Air-Drum Pad는 다음 두 가지 신호를 결합한다.

- 1 손끝 하강 속도: 손끝이 이미지상 아래 방향으로 빠르게 움직이는지 확인한다.
- 2 손가락 관절 각속도: 손가락 자체가 실제로 굽혀지거나 펴지는지 확인한다.

두 조건을 동시에 만족할 때만 타격으로 인정함으로써 음악적 입력 의도를 더 안정적으로 검출한다.

### 1.1 기여점

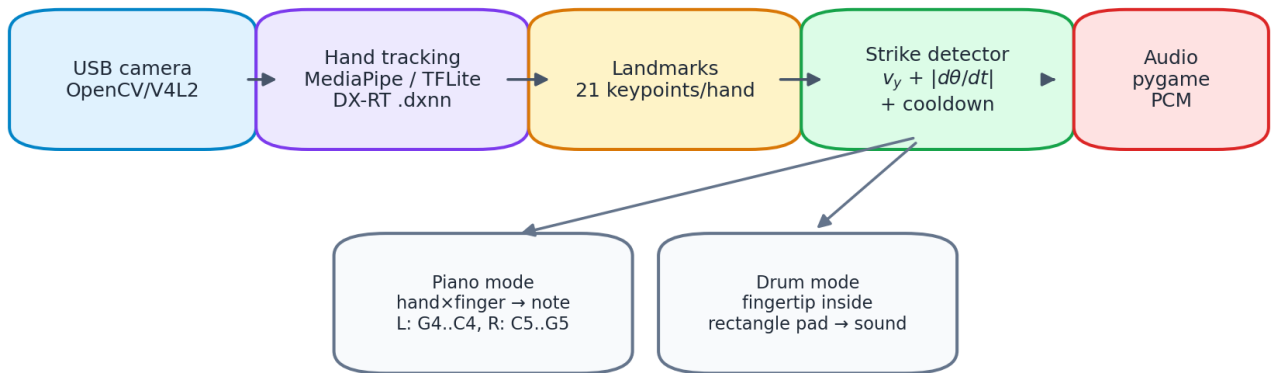
본 프로젝트의 주요 기여는 다음과 같다.

- 실시간 손 랜드마크 기반 드럼 및 피아노 비접촉 음악 인터페이스 구현
- 손끝 속도와 관절 각속도를 결합한 공통 타격 검출기 구현
- 직사각형 패드 기반 드럼 인터페이스와 JSON 기반 커스텀 패드 레이아웃 지원
- 왼손 엄지 G4, 왼손 소지 C4, 오른손 C5–G5를 반영한 피아노 기본 매핑 설정
- 런타임 및 문서에 사용할 드럼/피아노 매핑 다이어그램 자동 생성
- 시스템 구조, 타격 로직, 백엔드 지연 비교를 위한 보고서용 그림 자동 생성
- 매핑, 타격 검출, 패드 검증, ROI 변환, 벤치마크 헬퍼에 대한 반복 가능한 테스트 체계 구축

## 2. 시스템 개요

그림 1은 전체 런타임 파이프라인을 나타낸다. USB 카메라에서 프레임을 받아 손 추적 백엔드로 처리하고, 각 손의 21개 랜드마크를 얻은 뒤, 타격 검출기를 통해 피아노 음 또는 드럼 패드 소리로 매핑한다. 최종적으로 사전 생성된 PCM 오디오 버퍼를 재생한다.

### AI Air-Drum Pad Runtime Pipeline



The same vision and strike detector support both musical interfaces; only mapping changes.

그림 1. tools/gen\_report\_figures.py로 생성한 런타임 파이프라인.

동일한 비전 파이프라인과 타격 검출기가 두 모드에 공통으로 사용된다. 차이는 마지막 매핑 계층에 있다.

- 피아노 모드:  $\square \text{ ID} \times \square \square \square \text{ ID} \rightarrow \square \square$
- 드럼 모드:  $\square \square \square \square \square \square \square \square \rightarrow \square \square \square$

### 2.1 구현된 모드

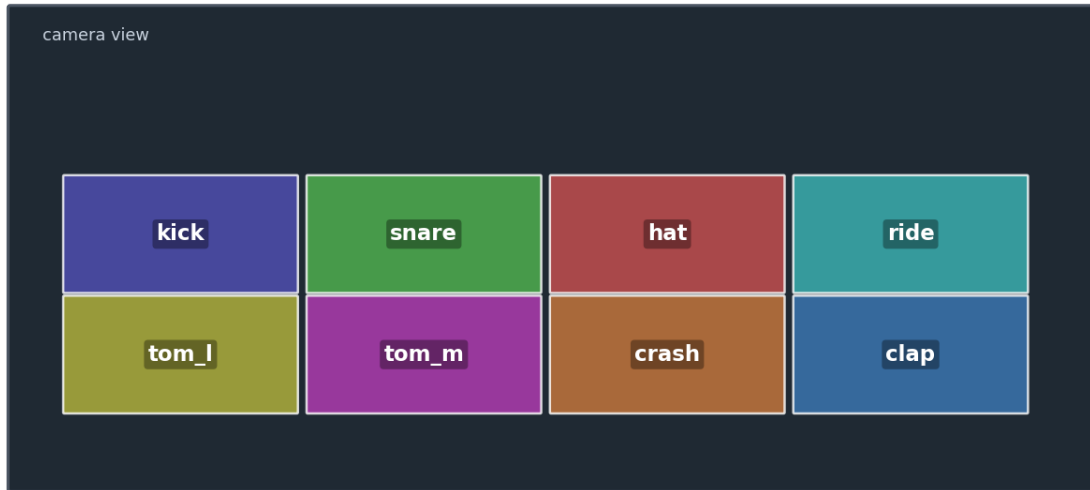
#### 드럼 패드 모드

최종 드럼 모드는 더 이상 손가락별 고정 드럼 매핑을 사용하지 않는다. 대신 화면에 표시되는 정규화 직사각형 패드를 사용한다. 어떤 손가락이든 패드 안에서 타격 조건을 만족하면 해당 패드의 드럼 소리를 재생한다. 기본 레이아웃은 kick, snare, hat, ride, tom\_l, tom\_m, crash, clap의 8개 패드로 구성된다.

## Drum Pads — Default Layout

*8 on-screen rectangles: move any fingertip into a pad and strike downward*

```
$ python3 main.py --camera 0
```



Any tracked fingertip can hit any rectangle; drum mode is no longer a fixed per-finger map.

그림 2. tools/gen\_instrument\_diagrams.py로 생성한 기본 드럼 패드 레이아웃.

다음 그림은 커스텀 패드 레이아웃에 사용할 수 있는 전체 내장 드럼 사운드 키를 보여준다.

## All Available Drum Sounds

*16 built-in percussion samples usable as pad sound keys*

```
$ python3 main.py --drum-pads pads.example.json --camera 0
```



Use these sound keys in pads.example.json or another --drum-pads layout.

그림 3. tools/gen\_instrument\_diagrams.py로 생성한 내장 드럼 사운드 키 목록.

피아노 모드

피아노 모드는 손과 손가락 ID를 10개의 음 슬롯에 매핑한다. 최종 기본 배치는 다음과 같다.

| 손            | 엄지 | 검지 | 중지 | 약지 | 소지 |
|--------------|----|----|----|----|----|
| 왼손 / Hand 0  | G4 | F4 | E4 | D4 | C4 |
| 오른손 / Hand 1 | C5 | D5 | E5 | F5 | G5 |

이 배치는 사용자가 요구한 왼손 방향을 반영한다. 왼손 엄지는 왼손에서 가장 높은 음인 G4이고, 왼손 소지는 가장 낮은 음인 C4이다. 오른손은 C5에서 G5까지 상승한다.

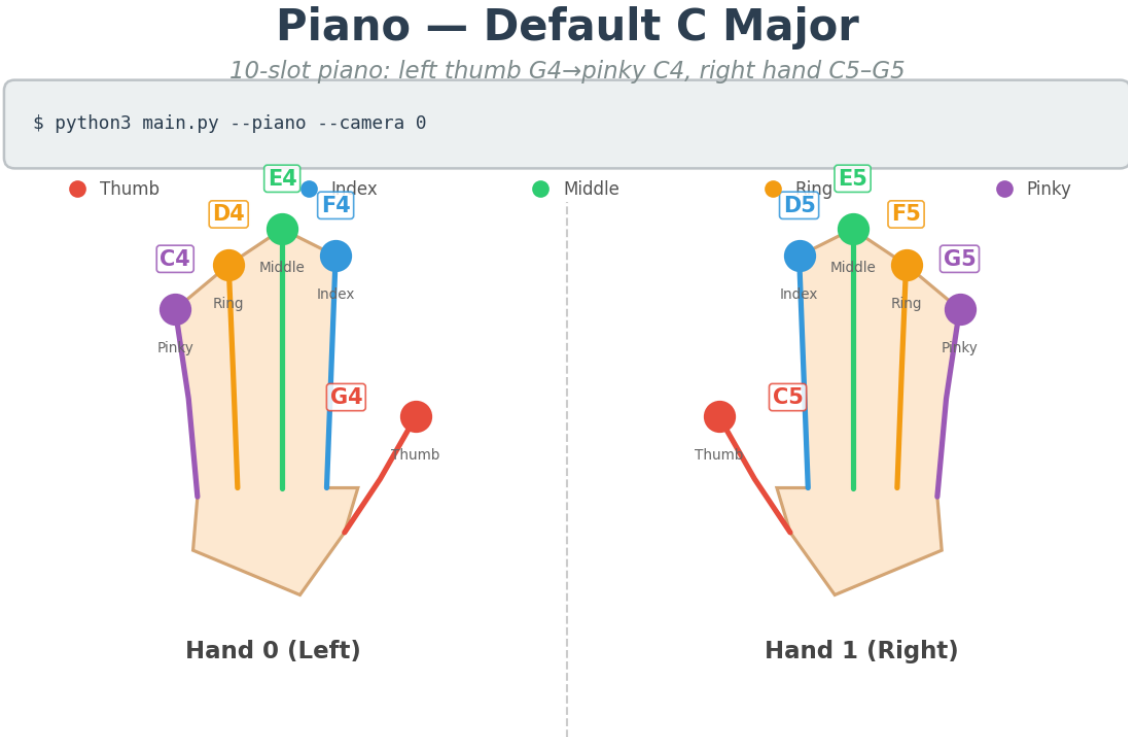


그림 4. tools/gen\_instrument\_diagrams.py로 생성한 기본 피아노 레이아웃.

커스텀 피아노 JSON 예시도 동일한 기본 슬롯 순서를 이용해 생성된다.

# Piano — Custom JSON Example

Fixed 10-note layout from `instruments.piano.example.json`

```
$ python3 main.py --piano --instruments instruments.piano.example.json --camera 0
```

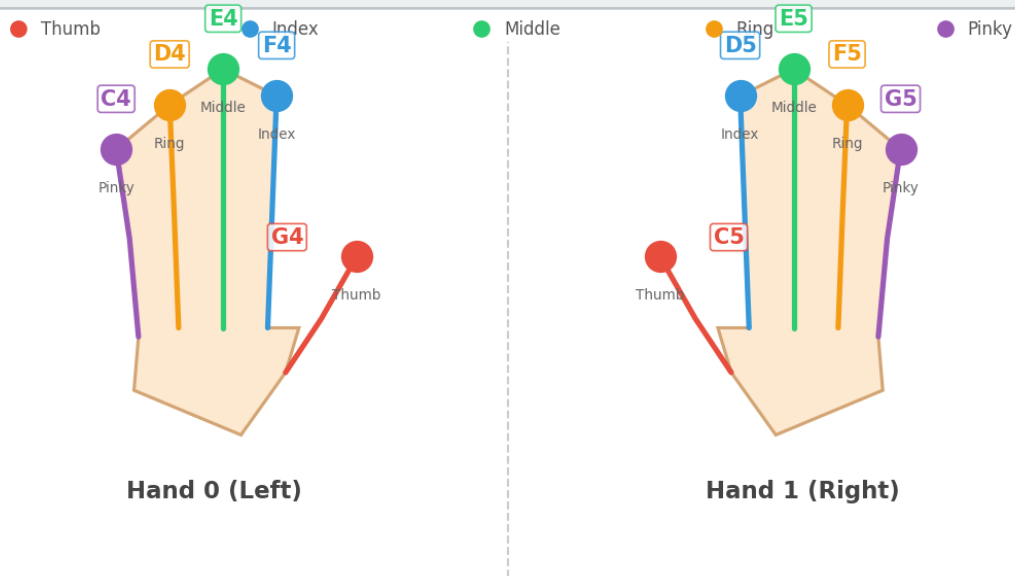


그림 5. `tools/gen_instrument_diagrams.py`로 생성한 피아노 커스텀 레이아웃 예시.

## 2.2 수동 캡처가 필요한 그림

다음 그림은 실제 하드웨어 실행 화면을 수동으로 캡처해야 하므로 FILLME로 남겨둔다.

[FILLME image placeholder: FILLME: 카메라 영상, 패드 오버레이, 사이드바, 타격 목록이 보이는 최종 드럼 모드 전체 화면.]

[FILLME image placeholder: FILLME: 카메라 영상, 손가락 궤적, 사이드바, 타격 목록이 보이는 최종 피아노 모드 전체 화면.]

## 3. 배경 및 관련 연구

AI Air-Drum Pad는 제스처 기반 음악 인터페이스, 실시간 손 추적, 임베디드 엣지 AI가 만나는 지점에 위치한다. 기존의 제스처 악기는 깊이 카메라, 웨어러블 센서, 관성 센서, 또는 영상 기반 포즈 추정을 사용해 왔다. 웨어러블 센서 방식과 비교하면 카메라 기반 시스템은 장착 장비가 필요 없고 접근성이 높다. 그러나 카메라 시스템은 접촉 이벤트가 없기 때문에 시각적 움직임만으로 사용자의 의도를 추정해야 한다.

본 프로젝트는 현대적인 단일 RGB 카메라 기반 손 포즈 추정과 유사한 접근을 사용한다. Palm 또는 hand detector가 손 위치를 찾고, landmark network가 21개 핵심점을 추정하며, 응용 로직이 핵심점의 시간 변화로부터 음악적 제스처를 판단한다. 또한 저지연 음악 상호작용에는 인식 정확도뿐 아니라 오디오 응답성이 중요하므로 CPU/NPU 배치에 따른 지연 특성도 함께 살펴본다.

## 4. 방법

### 4.1 손 랜드마크 표현

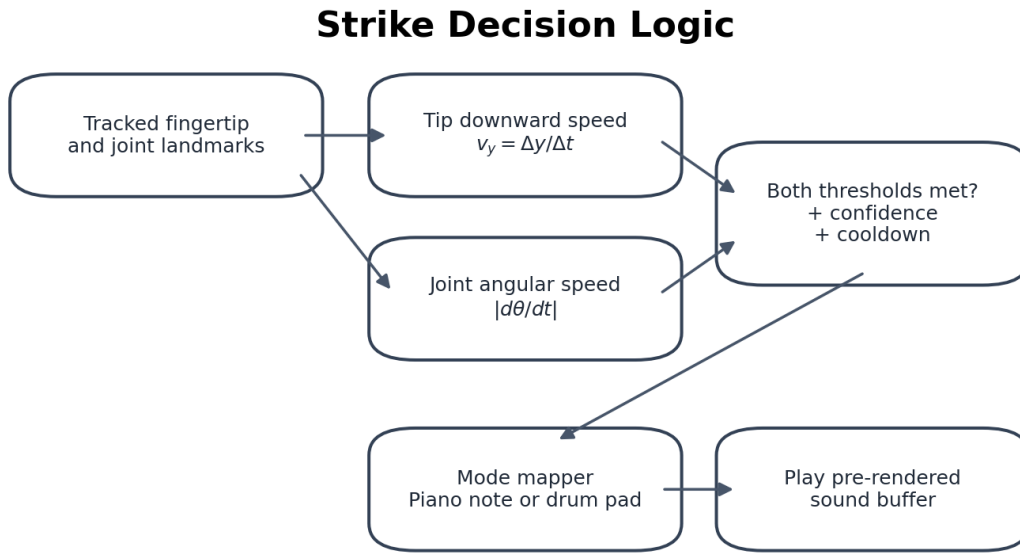
각 손은 21개의 정규화 랜드마크로 표현된다. 애플리케이션은 다음 다섯 손끝 인덱스를 사용한다.

| 손가락 | 랜드마크 인덱스 |
|-----|----------|
| 엄지  | 4        |
| 검지  | 8        |
| 중지  | 12       |
| 약지  | 16       |
| 소지  | 20       |

각 손가락에는 관절 각속도를 계산하기 위한 3점 체인이 정의되어 있다. 예를 들어 검지는 MCP-PIP-TIP 체인을 사용하고, 엄지는 엄지 IP 관절을 포함한 별도 체인을 사용한다.

## 4.2 타격 검출

그림 6은 타격 판단 로직을 보여준다.



A hit is emitted only when downward motion and finger articulation occur together, reducing false positives from arm-only motion.

그림 6. tools/gen\_report\_figures.py로 생성한 타격 판단 로직.

시간  $w(tw)$ 에서 손끝의 정규화  $y$ 좌표를  $w(y_tw)$ 라고 할 때, 손끝 하강 속도는 다음과 같이 근사한다.

$$w[v_y = \frac{y_t - y_{t-1}}{t - t_{t-1}}]. w]$$

이미지 좌표계에서  $y$ 는 아래로 갈수록 증가하므로 양의  $w(v_yw)$ 는 아래 방향 움직임을 의미한다. 선택된 손가락 관절의 각도를  $w(wtheta_tw)$ 라고 하면 관절 각속도는 다음과 같이 계산한다.

$$w[w\omega = \left| \frac{wtheta_t - wtheta_{t-1}}{t - t_{t-1}} \right|. w]$$

타격 후보는 다음 조건을 만족해야 한다.

$$w[v_y \geq \tau_v \text{ and } w\omega \geq \tau_\omega]. w]$$

이후 confidence filtering, 최소 변위 조건, 쿨다운 조건을 적용한다. 중지는 카메라 좌표상 각도 변화가 상대적으로 작게 나타나는 경우가 있어 내부 threshold scale을 낮추어 민감도를 보정한다.

## 4.3 드럼 패드 매핑

각 드럼 패드는 다음과 같은 정규화 직사각형으로 표현된다.

$$\mathbb{W}[(x_1, y_1, x_2, y_2), \mathbb{W} \text{quad } 0 \leq x_1 < x_2 \leq 1, \mathbb{W} \text{quad } 0 \leq y_1 < y_2 \leq 1. \mathbb{W}]$$

손끝 타격이 발생한 시점에 손끝 좌표가 특정 패드 내부에 있으면 해당 패드의 sound key를 반환한다. 쿨다운은 패드 label 단위로 적용되므로 서로 다른 패드는 빠르게 연속해서 연주할 수 있다.

커스텀 드럼 패드는 다음과 같은 JSON 형식으로 정의한다.

```
{
  "pads": [
    {"label": "kick", "sound": "kick", "x1": 0.05, "y1": 0.35, "x2": 0.29, "y2": 0.59, "color": [80, 80, 180]}
  ]
}
```

## 4.4 피아노 매핑

피아노 매핑은 다음 순서의 10개 슬롯을 사용한다.

```
[Hand 0 00, 00, 00, 00, 00,
 Hand 1 00, 00, 00, 00, 00]
```

최종 기본 튜플은 다음과 같다.

```
("G4", "F4", "E4", "D4", "C4", "C5", "D5", "E5", "F5", "G5")
```

이 매핑은 `drumkit_audio.py`와 `main.py`가 기본으로 읽는 `instruments.piano.example.json`에 반영되어 있다.

## 4.5 오디오 생성

시스템은 타격 시점에 오디오를 즉석 합성하지 않고, 미리 생성한 `pygame.mixer.Sound` 객체를 재생한다. 피아노 음은 약 0.5초 길이의 합성음으로 생성된다. 짧은 드럼 샘플도 사용자가 타격을 인지하기 쉽도록 길이를 늘렸다.

# 5. 구현

## 5.1 주요 파일

| 파일                                            | 역할                                              |
|-----------------------------------------------|-------------------------------------------------|
| <code>main.py</code>                          | 카메라 루프, 백엔드 선택, 모드 선택, 화면 표시, 오디오 트리거           |
| <code>hand_tracker.py</code>                  | MediaPipe, TFLite CPU, NPU, NPU-full 추적 백엔드 추상화 |
| <code>strike_detector.py</code>               | 손가락 타격 검출기, 패드 타격 검출기, 패드 JSON 검증               |
| <code>drumkit_audio.py</code>                 | 합성 드럼 샘플, 피아노 음 합성, 피아노 기본 슬롯                   |
| <code>tools/gen_instrument_diagrams.py</code> | 드럼/피아노 인터페이스 다이어그램 생성                           |
| <code>tools/gen_report_figures.py</code>      | 보고서용 구조 및 결과 그림 생성                              |
| <code>tools/export_report_documents.py</code> | Markdown 보고서를 PDF 및 DOCX로 변환                    |
| <code>tests/</code>                           | 타격 검출, 매핑, ROI, 벤치마크 헬퍼 단위 테스트                  |

## 5.2 런타임 백엔드

시스템은 다음 네 가지 백엔드를 지원한다.

| 백엔드          | Palm detection      | Hand landmark | 용도                                    |
|--------------|---------------------|---------------|---------------------------------------|
| cpu          | MediaPipe 내부        | MediaPipe 내부  | 간단한 CPU-only baseline                 |
| cpu-baseline | CPU TFLite          | CPU TFLite    | NPU 없이 동일 파이프라인 비교                    |
| npu-full     | CPU TFLite          | NPU .dxnn     | 정확한 palm 기반 파이프라인과 NPU hand inference |
| npu          | 없음 / dual-halves 근사 | NPU .dxnn     | 가장 빠른 저지연 근사 모드                       |

정확도를 위해서는 palm detection이 포함된 **npu-full**이 바람직하지만, 현재 CPU palm detection이 전체 지연의 대부분을 차지한다. 반면 **npu dual-halves** 모드는 palm detector를 제거해 빠르지만, 카메라 구도에 의존하는 근사 방식이므로 실제 설치 환경에서 검증해야 한다.

## 5.3 사용자 인터페이스

런타임 화면은 카메라 영상, 손/손가락 랜드마크 및 궤적, 모드별 오버레이, 최근 타격 목록, 그리고 매핑 다이어그램 사이드바를 포함한다. 드럼 모드에서는 카메라 영상 위에 직사각형 패드가 그려지고, 타격 시 짧게 밝게 표시된다. 피아노 모드에서는 손가락별 음 매핑과 최근 음 이벤트를 확인할 수 있다.

## 6. 실험 방법

### 6.1 소프트웨어 검증

현재 품질 검사는 다음 명령으로 수행한다.

```
RUN_BENCH_SMOKE=0 ./scripts/check_quality.sh
```

이 명령은 Python syntax check, 단위 테스트, palm decode 테스트를 실행한다. 모델 파일 또는 하드웨어 의존성이 없는 환경에서는 benchmark smoke test를 의도적으로 생략할 수 있다.

### 6.2 백엔드 지연 측정

백엔드 평가는 실시간 비전 악기에서 서로 섞여 보이기 쉬운 세 가지 질문을 분리하기 위해 설계하였다.

- 1 단순 CPU-only 손 추적 기준선은 얼마나 빠른가?
- 2 전체 지연 중 palm detection과 hand landmark inference가 각각 얼마나 차지하는가?
- 3 NPU가 실제로 NPU에 배치한 hand landmark 단계에서 이득을 주는가, 그리고 더 빠른 근사 경로는 어떤 정확도 위험을 가지는가?

따라서 하나의 프로토타입만 측정하지 않고 네 가지 런타임 구성을 비교하였다.

| 구성              | 평가 대상                                                               | 기준선이 필요한 이유                                                                    |
|-----------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------|
| cpu / MediaPipe | MediaPipe 내장 palm/hand stack을 사용하는 CPU-only 참조 경로                   | 모델 변환 의존성 없이 음악 로직이 동작하는지 확인하고, 가장 단순하고 안정적인 기준선을 제공한다                         |
| cpu-baseline    | Palm TFLite + hand landmark TFLite를 모두 CPU에서 실행                     | <b>npu-full</b> 과 동일한 palm→ROI→hand 구조이므로 hand inference만 NPU로 바꿨을 때의 효과를 분리한다 |
| npu-full        | Palm TFLite는 CPU, hand landmark .dxnn은 NPU에서 실행                     | 신뢰 가능한 float32 palm detection을 유지하면서 정확한 하이브리드 edge-AI 파이프라인을 평가한다             |
| npu dual-halves | Palm detector 없이 화면을 두 손 영역으로 근사 분할하고 hand landmark .dxnn을 NPU에서 실행 | Palm detection을 제거했을 때의 저지연 하한을 추정한다. 라이브 데모에는 유용하지만 손 위치 변화에는 덜 강건하다          |

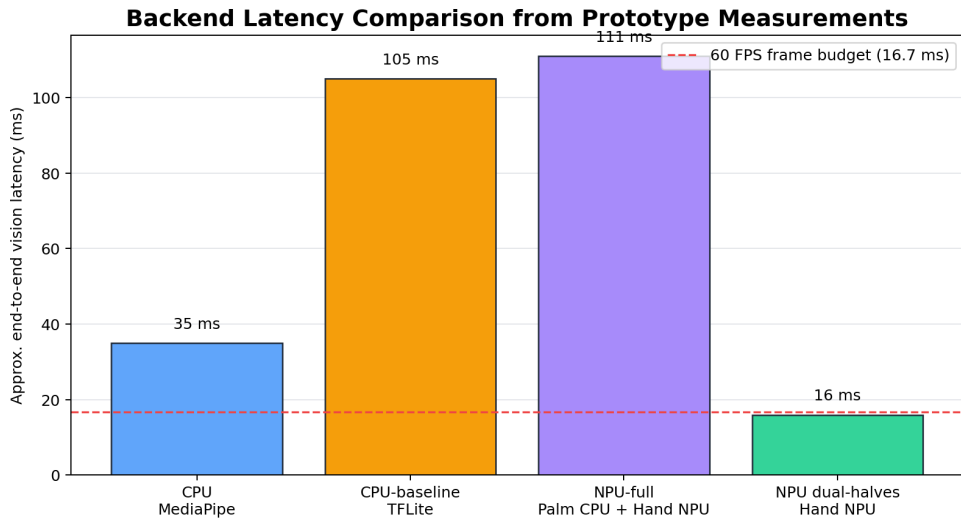
본 보고서의 주요 지연 지표는 비전 루프 지연이다. 이는 카메라 프레임 처리, palm/hand inference, ROI 처리, landmark 산출까지의 시간이며, 실제 스피커에서 소리가 나기까지의 완전한 acoustic E2E latency는 아니다. 상호작용 목표는 60 FPS에 해당하는 16.7 ms 프레임 예산이며, 최종 음향 지연은 별도 실험으로 보고해야 한다.



기록된 프로토타입 측정 환경은 다음과 같다.

| 항목                | 값                                  |
|-------------------|------------------------------------|
| Board             | Orange Pi 5 Plus (RK3588, aarch64) |
| OS / Python       | Linux, Python 3.10.12              |
| DX-RT / dx_engine | 1.1.4                              |
| DX-COM            | 외부 compile server의 v2.1.0-rc.4     |
| Camera            | USB camera, 640×480                |
| Offline dataset   | dataset/frame_*.png의 90개 캡처 프레임    |

Palm detector도 NPU .dxnn 후보로 시험하였다. NPU palm 모델은 약 12 ms로 빠르지만 INT8 양자화가 score head를 손상시켰고, ONNX-to-NPU score correlation이 약 -0.11로 나타나 실제 손 검출에 사용할 수 없었다. 따라서 정확한 palm 기반 구성은 모두 CPU TFLite palm detection을 사용하며, 이 단계가 주요 지연 병목으로 남는다.



Four runtime configurations are compared to isolate palm detection, hand-landmark inference, and NPU dispatch effects; final audio latency still requires manual high-speed-camera capture.

그림 7. tools/gen\_report\_figures.py로 생성한 백엔드 지연 비교.

### 6.3 추가로 필요한 수동 측정

완전한 학술 평가는 실제 물리적 상호작용 루프의 수동 측정을 포함해야 한다. 아래 절차를 보고서 본문에 명시해 두면, 추후 측정값을 별도 노트가 아니라 본 보고서에 직접 삽입할 수 있다.

- 1 E2E 오디오 지연: 고속 카메라 또는 동기화된 오디오/비디오 장비로 연주자의 손과 스피커 출력을 동시에 기록한다. 각 타격에 대해 하강 타격이 시작되는 비디오 프레임과 스피커 파형의 첫 onset을 찾고 다음 값을 계산한다. 여기서  $\Delta t = t_{\text{audio}} - t_{\text{video}}$ 이다. 모드별 최소 30회 이상 측정하고 평균, 표준편차, P95 지연을 보고한다.
- 2 타격 정확도: 드럼 및 피아노 모드에서 메트로놈 기반 반복 실험을 수행한다. True positive는 허용 시간창 안에서 검출된 의도 타격, false negative는 의도 타격이 있었지만 이벤트가 없는 경우, false positive는 의도하지 않은 이벤트 또는 중복 이벤트로 정의한다. TP, FP, FN, precision, recall, 주요 실패 사례를 보고한다.
- 3 실험 로그: git commit, backend, camera resolution, model path/checksum, vy\_trigger, joint\_dps, cooldown, 조명 조건, 그리고 t, frame\_id, infer\_ms, hand\_id, finger\_id, pad/note, trigger 같은 raw event log field를 보관한다.

다음 그림은 실제 하드웨어 캡처가 필요하므로 FILLME로 남겨둔다.

[FILLME image placeholder: FILLME: 손 동작과 스피커/오디오 파형을 동시에 기록하는 E2E 지연 측정 셋업.]

[FILLME image placeholder: FILLME: 피아노 및 드럼 모드 타격 정확도 실험 결과 그래프 또는 표.]

## 7. 결과

### 7.1 기능 결과

최종 구현은 다음 기능 목표를 만족한다.

- 라이브 카메라 입력에서 손 랜드마크를 검출한다.
- 손끝 속도와 관절 각속도를 사용해 의도적 타격을 검출한다.
- 손/손가락 ID 기반으로 피아노 음을 재생한다.
- 화면상 직사각형 패드 기반으로 드럼 소리를 재생한다.
- 커스텀 피아노 JSON 및 드럼 패드 JSON 레이아웃을 지원한다.
- 문서와 런타임 UI에 사용할 매핑 다이어그램을 자동 생성한다.

### 7.2 테스트 결과

최근 검증 결과는 다음과 같다.

| 테스트 그룹                  | 결과                           |
|-------------------------|------------------------------|
| Python syntax check     | 통과                           |
| 단위 테스트                  | 28/28 통과                     |
| Palm decode 테스트         | 15/15 통과                     |
| Dataset benchmark smoke | RUN_BENCH_SMOKE=0으로 의도적으로 생략 |

단위 테스트는 피아노 기본 매핑, 합성 오디오 길이, 패드 영역 생성, 패드 JSON 검증, instrument slot 검증, 타격 검출 threshold, 중지 민감도, 쿨다운 동작, ROI helper, benchmark helper, dataset capture indexing을 포함한다.

### 7.3 백엔드 성능 요약

아래 표는 네 가지 런타임 구성의 지연을 비교한다. `cpu`는 실용적인 참조 경로이고, `cpu-baseline`과 `npu-full`은 동일한 `palm→ROI→hand` 구조를 공유하므로 CPU hand inference와 NPU hand inference를 통제 비교할 수 있다. `npu dual-halves`는 정확도 동등 비교가 아니라 palm detection을 제거한 저지연 근사 경로이다.

| 백엔드                                  | Palm 단계 | Hand 단계   | 2손 기준 대략적 전체 지연 | 해석                                                                  |
|--------------------------------------|---------|-----------|-----------------|---------------------------------------------------------------------|
| <code>cpu</code> (MediaPipe)         | ~15 ms  | ~10 ms    | ~35 ms          | 간단한 CPU-only 기능 기준선, 추가 모델 파일 불필요                                   |
| <code>cpu-baseline</code> (TFLite)   | ~95 ms  | ~5 ms     | ~105 ms         | <code>npu-full</code> 과 동일한 2단 구조, hand landmark 가속 효과를 분리하기 위한 기준선 |
| <code>npu-full</code> (TFLite + NPU) | ~95 ms  | ~8 ms     | ~111 ms         | 정확한 하이브리드 경로이지만 CPU palm detection이 전체 지연을 지배                       |
| <code>npu dual-halves</code>         | 0 ms    | ~8 ms × 2 | ~16 ms          | 60 FPS 예산에 가까운 가장 빠른 경로, 단 palm detection 없이 손 위치를 근사               |

이 결과는 두 가지를 보여준다. 첫째, 정확한 파이프라인에서 CPU palm detection이 지배적이면 hand landmark만 NPU로 옮겨서는 전체 지연을 충분히 낮추기 어렵다. 둘째, `dual-halves` 경로는 라이브 데모에 유용하지만 손 위치가 안정적이라는 가정을 가지므로 palm 기반 구성과 정확도 동등 비교로 해석해서는 안 된다.

### 7.4 오프라인 데이터셋 벤치마크 요약

90프레임 오프라인 벤치마크는 구조적으로 대응되는 `cpu-baseline`과 `npu-full`을 같은 입력 프레임에서 비교하기 위해 수행되었다. 이 실험은 라이브 카메라 변동을 제거하고 프레임별 처리 지연과 landmark 일치도를 확인한다.

| 구성                                                                  | 평균 지연       | P95 지연       | 세부 프로파일                                 | 해석                                             |
|---------------------------------------------------------------------|-------------|--------------|-----------------------------------------|------------------------------------------------|
| <code>cpu-baseline</code> , palm every frame                        | 84.40 ms    | 86.78 ms     | palm 39.26 ms + hand 44.80 ms           | 2단 파이프라인의 CPU TFLite 기준                        |
| <code>npu-full</code> , palm every frame                            | 50.32 ms    | 54.52 ms     | palm 40.93 ms + hand 9.13 ms            | 같은 palm/ROI 경로에서 hand inference만 NPU로 이동       |
| <code>npu-full</code> ,<br>--palm-redetect-every 5 (20-frame smoke) | 15.29 ms    | 50.96 ms     | palm frames 3/20, tracking frames 17/20 | palm skip의 지연 개선 가능성을 보이거나 drift와 타격 정확도 검증 필요 |
| <code>npu-full</code> , --async-palm smoke                          | 10-20 ms 범위 | 입력 pacing 의존 | asynchronous palm refresh + tracking    | 실험 경로이며 live 안정성 검증 필요                         |

`cpu-baseline`과 `npu-full`의 통제 비교에서는 NPU가 TFLite-style 파이프라인의 hand landmark 시간을 44.80 ms에서 9.13 ms로 줄였다. 그러나 palm detection이 여전히 약 40 ms를 차지하므로, palm 빈도를 줄이거나 palm detection을 가속하지 않으면 정확한 파이프라인은 60 FPS 목표를 넘기 어렵다.

Landmark 정확도는 CPU-baseline을 기준으로 `npu-full` landmark를 정규화 이미지 좌표에서 비교하여 계산하였다.

| 손     | 매칭 프레임 | 21점 평균 오차 | 손끝 평균 오차 | 평균 max 오차 | 최대 max 오차 |
|-------|--------|-----------|----------|-----------|-----------|
| Right | 90     | 0.0270    | 0.0336   | 0.0532    | 0.1593    |
| Left  | 83     | 0.0256    | 0.0353   | 0.0457    | 0.0734    |

이후 NPU landmark의 체계적 편향을 줄이기 위해 affine correction 모델을 학습하였다. 같은 90프레임 training set에서 보정 후 오차는 다음과 같다.

| 손     | 매칭 프레임 | 21점 평균 오차 | 손끝 평균 오차 | 평균 max 오차 | 최대 max 오차 |
|-------|--------|-----------|----------|-----------|-----------|
| Right | 90     | 0.0102    | 0.0112   | 0.0223    | 0.1701    |
| Left  | 83     | 0.0092    | 0.0090   | 0.0193    | 0.0437    |

이 보정은 training capture에서는 평균 오차와 손끝 오차를 낮추지만, 조명, 손 크기, 손 자세, 카메라 거리가 다른 별도 hold-out capture에서 검증되기 전까지 일반화된 정확도 개선으로 주장해서는 안 된다.

## 8. 논의

### 8.1 두 가지 움직임 신호의 필요성

가상 악기는 손이 움직였다는 이유만으로 소리를 내면 안 된다. 손끝 하강 속도와 관절 각속도를 동시에 요구하면, 손 전체 이동과 실제 손가락 타격을 더 잘 구분할 수 있다. 이는 손가락별 음이 중요한 피아노 모드와 패드 영역을 지나갈 수 있는 드럼 모드 모두에서 중요하다.

### 8.2 패드 기반 드럼 모드의 장단점

초기 드럼 모드는 손가락별 고정 매핑을 사용했지만, 최종 모드는 직사각형 패드 기반으로 바뀌었다. 이 방식은 실제 드럼 패드 컨트롤러와 유사해 직관적이며, 사용자가 어떤 손가락으로도 원하는 패드를 칠 수 있다. 반면 패드 위치가 카메라 구도와 화면 좌표에 의존하므로 시각적 피드백과 레이아웃 조정 기능이 중요하다.

### 8.3 피아노 매핑 규칙

최종 피아노 매핑은 왼손 엄지가 왼손에서 가장 높은 음인 G4, 왼손 소지가 가장 낮은 음인 C4가 되도록 구성되었다. 오른손은 C5부터 G5까지 상승한다. 이 규칙은 카메라를 향한 손 그림과 사용자의 의도한 실제 연주 방향을 반영한다.

### 8.4 엣지 AI 배치의 트레이드오프

NPU는 hand landmark inference를 가속할 수 있지만, 전체 파이프라인에는 palm detection도 필요하다. 위 실험에서 NPU palm 후보는 빠르지만 INT8 양자화가 detection score를 손상시켜 실제 검출에 사용할 수 없었고, 정확한 파이프라인에서는 CPU TFLite palm detection을 유지하였다. 따라서 npu-full의 전체 지연은 기대보다 크며, 향후에는 palm detector 양자화 개선, 대체 detector, 또는 drift를 제어하는 palm skip 전략이 필요하다.

## 9. 한계

현재 프로토타입에는 다음 한계가 있다.

- 최종 E2E 오디오 지연 측정이 아직 포함되어 있지 않다.
- 정식 사용자 연구가 수행되지 않았다.
- 템포 제어 환경에서의 타격 정확도 표가 아직 없다.
- 단일 RGB 카메라를 사용하므로 깊이 모호성이 존재한다.
- 조명, 배경, 손 자세가 landmark 안정성에 영향을 줄 수 있다.
- 정확한 npu-full 경로는 CPU palm detection에 의해 지연이 제한된다.
- 물리적 촉각 피드백이 없어 실제 악기와 연주감이 다를 수 있다.

## 10. 향후 연구

향후 작업은 다음과 같다.

- 1 FILLME로 남겨둔 최종 라이브 실행 화면을 캡처한다.
- 2 고속 카메라와 오디오 파형을 이용해 E2E 오디오 지연을 측정한다.
- 3 여러 템포에서 피아노/드럼 모드 타격 정확도를 측정한다.
- 4 사용자별 패드 위치, handedness, strike threshold 보정 기능을 추가한다.
- 5 NPU에 적합한 palm detector 대안을 조사한다.
- 6 외부 악기와 연결할 수 있도록 MIDI 출력을 추가한다.
- 7 평가 실험을 위한 시각/청각 메트로놈을 추가한다.
- 8 소규모 사용자 연구를 통해 연주 가능성과 사용성을 평가한다.

## 11. 결론

AI Air-Drum Pad는 카메라 기반 손 추적을 이용해 손 움직임을 드럼 및 피아노 이벤트로 변환하는 비접촉 음악 인터페이스이다. 최종 구현은 패드 기반 드럼 모드, 정정된 피아노 음 매핑, 사전 렌더링 오디오 재생, 자동 생성 문서 그림, 반복 가능한 테스트 스위트를 포함한다. 본 프로젝트는 엣지 AI 손 추적이 표현력 있는 비접촉 음악 상호작용을 지원할 수 있음을 보여준다. 동시에 안정적인 타격 검출, 손 위치 추정, 오디오 지연 측정, NPU 파이프라인 최적화가 실제 악기 수준의 사용성을 위해 중요하다는 점을 확인하였다.

## 감사의 글

FILLME: 수업명, 지도교수, 연구실, 하드웨어 제공자, 협업자 등을 기입한다.

# 참고문헌

[1] AI Air-Drum Pad 저장소 산출물, `git@github.com:GW-arch/deepx.git`의 `air_drum_pad/`: 본 self-contained 보고서 작성에 사용한 소스 코드, 테스트, 생성 그림, 악기 이미지, 기록된 벤치마크 산출물. [2] MediaPipe Hands 및 hand landmark model family: CPU MediaPipe 및 TFLite 기반 파이프라인에서 사용. [3] OpenCV: 카메라 입력, 화면 표시, drawing에 사용. [4] pygame: 사전 렌더링 PCM sound buffer의 저지연 재생에 사용. [5] DeepX DX-RT / DX-COM toolchain: `.dxnn` NPU hand landmark inference 실험에 사용.

## 부록 A. 재현 명령

인터페이스 다이어그램 생성:

```
cd air_drum_pad
python3 tools/gen_instrument_diagrams.py
```

보고서용 그림 생성:

```
cd air_drum_pad
python3 tools/gen_report_figures.py
```

보고서 DOCX/PDF 생성:

```
cd air_drum_pad
python3 tools/export_report_documents.py
```

품질 검사:

```
cd air_drum_pad
RUN_BENCH_SMOKE=0 ./scripts/check_quality.sh
```

기본 모드 실행:

```
cd air_drum_pad
python3 main.py --camera 0
python3 main.py --piano --camera 0
```

커스텀 드럼 패드 실행:

```
cd air_drum_pad
python3 main.py --camera 0 --drum-pads pads.example.json
```

## 부록 B. 주요 경로

| 경로                                            | 설명                         |
|-----------------------------------------------|----------------------------|
| <code>main.py</code>                          | 메인 런타임 애플리케이션              |
| <code>strike_detector.py</code>               | 타격 검출기 및 패드 검출기            |
| <code>drumkit_audio.py</code>                 | 오디오 및 피아노 음 정의             |
| <code>pads.example.json</code>                | 커스텀 드럼 패드 레이아웃 예시          |
| <code>instruments.piano.example.json</code>   | 기본/커스텀 피아노 음 예시            |
| <code>tools/gen_instrument_diagrams.py</code> | 모드별 인터페이스 다이어그램 생성         |
| <code>tools/gen_report_figures.py</code>      | 보고서용 그림 생성                 |
| <code>tools/export_report_documents.py</code> | Markdown 보고서를 DOCX/PDF로 변환 |
| <code>docs/figures/</code>                    | 보고서용 생성 그림                 |
| <code>instruments/</code>                     | 런타임 및 문서용 매핑 이미지           |